

Here is your emergency Docker cheat sheet. It is organized by real-world crisis scenarios so you can quickly find the exact command needed to inspect, stop, restart, or fix a malfunctioning container.



1. Emergency Container Control

When a container is frozen, behaving wildly, or needs an immediate reset.

- **Stop a misbehaving container gracefully:**
Bash
`docker stop <container_name>`
Example: `docker stop motioneye`
- **Force-kill a completely locked-up container instantly:**
Bash
`docker kill <container_name>`
- **Start a stopped container:**
Bash
`docker start <container_name>`
- **Quickly restart a glitching container:**
Bash
`docker restart <container_name>`
Example: `docker restart homeassistant`
- **Pause all processes inside a container (freezes it in place without stopping it):**
Bash
`docker pause <container_name>`
- **Unpause a frozen container:**
Bash
`docker unpause <container_name>`



2. Checking Container Status

When you need to see what is currently running, crashed, or failing health checks.

- **List only active, running containers:**
Bash
`docker ps`
- **List all containers (including crashed, stopped, or exiting ones):**
Bash
`docker ps -a`
- **Find out why a container exited or check its exact error status:**

```
Bash
docker ps -a --format "table {{.Names}}\t{{.Status}}\t{{.State}}"
```

3. Inspecting Logs & Troubleshooting

When a container refuses to start or works incorrectly and you need to see the errors.

- **View the last 50 lines of logs to quickly catch errors:**

```
Bash
docker logs --tail 50 <container_name>
# Example: docker logs --tail 50 nextcloud-app-1
```

- **Follow logs in real-time (live-stream incoming errors):**

```
Bash
docker logs -f <container_name>
```

- **Follow live logs and only look for specific error keywords:**

```
Bash
docker logs -f <container_name> 2>&1 | grep -i "error"
```

- **View logs from a specific timestamp (e.g., the last 30 minutes):**

```
Bash
docker logs --since 30m <container_name>
```

4. Resource Monitoring (RAM/CPU/Disk)

When your server is running incredibly hot, slowing down, or freezing due to high resource usage.

- **Live resource monitor (CPU, RAM, Net I/O, Block I/O for all containers):**

```
Bash
docker stats
```

- **Check resource usage for one specific container:**

```
Bash
docker stats <container_name>
```

- **Check how much physical host disk space Docker is consuming:**

```
Bash
docker system df
```

- **Emergency Disk Space Cleanup (Deletes stopped containers, unused networks, and dangling image caches):**

```
Bash
docker system prune -f
```

- **Deep Disk Space Cleanup (Deletes everything not actively running—use with caution):**

Bash

```
docker system prune -a --volumes -f
```



5. Executing Commands Inside Containers

When you need to jump inside a container to check its local files, test network connections, or modify internal configs.

- **Open an interactive terminal inside a container:**

Bash

```
docker exec -it <container_name> /bin/bash
```

If bash doesn't work, try alpine sh:

```
docker exec -it <container_name> /bin/sh
```

- **Run a quick network test from inside a container (e.g., testing database connection):**

Bash

```
docker exec -it <container_name> ping <target_ip_or_container>
```

- **Check the mapped configurations and storage paths of a container:**

Bash

```
docker inspect <container_name>
```



6. Docker Compose Emergencies

If your containers were built using a docker-compose.yml file, run these commands inside the folder containing that file.

- **Take down the entire stack and remove its private network bridges:**

Bash

```
docker compose down
```

- **Start the stack in the background and force-rebuild any modified settings:**

Bash

```
docker compose up -d --build
```

- **Restart a single service inside the compose file without disturbing the others:**

Bash

```
docker compose restart <service_name_in_yaml>
```

- **View combined live logs for the entire compose stack:**

Bash

```
docker compose logs -f
```